

« Power BI te donne des boutons. Python te donne le contrôle. »

LIBRAIRIES DU COURS

pandas · numpy · scipy · matplotlib · seaborn · plotly

CHAPITRE I – EXTRACTION DES DONNÉES

— DEPUIS DES FICHIERS PLATS —

Lire un CSV avec contrôle total :

```
df = pd.read_csv("fichier.csv", encoding="utf-8", sep=";",  
                parse_dates=["date"], dtype={"id": str, "code_postal": str},  
                na_values=["N/A", "null", "-", ""], thousands=".",  
                decimal=".", skiprows=0, nrows=None)
```

Détecter automatiquement l'encodage quand inconnu :

```
import chardet  
with open("fichier.csv", "rb") as f:  
    enc = chardet.detect(f.read())["encoding"]  
df = pd.read_csv("fichier.csv", encoding=enc)
```

Lire un Excel avec contrôle du périmètre :

```
df = pd.read_excel("fichier.xlsx", sheet_name="Ventes",  
                  header=0, skiprows=0, usecols="A:H",  
                  parse_dates=["date"], dtype={"ref": str})
```

Lire toutes les feuilles d'un Excel en une passe :

```
sheets = pd.read_excel("rapport.xlsx", sheet_name=None)  
df = pd.concat(sheets.values(), ignore_index=True)
```

Lire un JSON plat ou imbriqué :

```
df = pd.read_json("data.json")  
df = pd.json_normalize(data, record_path=["commandes"],  
                       meta=["client_id", "date"])
```

Lire plusieurs fichiers d'un dossier en une passe :

```
from pathlib import Path  
df = pd.concat([pd.read_csv(f) for f in  
                Path("data/").glob("*.csv")], ignore_index=True)
```

Lire un gros fichier par chunks sans saturer la mémoire :

```
chunks = []  
for chunk in pd.read_csv("big.csv", chunksize=50000):  
    chunks.append(chunk[chunk["statut"] == "actif"])  
df = pd.concat(chunks, ignore_index=True)
```

— DEPUIS DES BASES DE DONNÉES —

Connexion universelle avec SQLAlchemy :

```
from sqlalchemy import create_engine, text  
engine = create_engine(  
    "postgresql+psycopg2://user:pass@host:5432/dbname")
```

Lire en DataFrame directement :

```
df = pd.read_sql(  
    text("SELECT * FROM ventes WHERE date >= :d"),  
    con=engine, params={"d": "2024-01-01"})
```

Écrire un DataFrame en base :

```
df.to_sql("ventes_clean", engine, if_exists="append",  
         index=False, method="multi", chunksize=1000)
```

SQLite sans serveur, idéal pour les projets légers :

```
import sqlite3
conn = sqlite3.connect("local.db")
df = pd.read_sql("SELECT * FROM ventes", conn)
df.to_sql("ventes_clean", conn, if_exists="replace",
        index=False)
conn.close()
```

Toujours utiliser des requêtes paramétrées. Jamais de f-string dans une requête SQL. C'est une faille de sécurité et une source de bugs sur les caractères spéciaux.

— DEPUIS DES APIS

Appel simple avec gestion d'erreur :

```
import requests, time, os
def call_api(url, params=None):
    headers = {"Authorization": f"Bearer {os.getenv('API_KEY')}}"
    r = requests.get(url, headers=headers,
        params=params, timeout=30)
    r.raise_for_status()
    return r.json()
```

Pagination automatique jusqu'à la dernière page :

```
def get_all(base_url):
    results, page = [], 1
    while True:
        data = call_api(base_url, {"page": page, "limit": 100})
        if not data.get("items"):
            break
        results.extend(data["items"])
        page += 1
        time.sleep(0.3)
    return pd.DataFrame(results)
```

Retry avec backoff exponentiel sur erreur réseau :

```
from requests.exceptions import RequestException
def safe_call(url, max_retries=3):
    for i in range(max_retries):
        try:
            return requests.get(url, timeout=30).json()
        except RequestException:
            if i == max_retries - 1: raise
            time.sleep(2 ** i)
```

— DEPUIS DU WEB (SCRAPING)

Extraire un tableau HTML directement en DataFrame :

```
tables = pd.read_html("https://exemple.com/tableau",
    header=0, thousands=".", decimal=",")
df = tables[0]
```

Scraping HTML structuré avec BeautifulSoup :

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(requests.get(url).text, "html.parser")
rows = [{"nom": tr.select_one(".nom").text.strip(),
    "valeur": tr.select_one(".val").text.strip()}
    for tr in soup.select("table tr")[1:]]
df = pd.DataFrame(rows)
```

Scraping JavaScript avec Playwright :

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    br = p.chromium.launch(headless=True)
    page = br.new_page()
    page.goto(url)
    page.wait_for_selector(".data-loaded")
    html = page.content()
    br.close()
df = pd.read_html(html)[0]
```

Vérifier robots.txt avant tout scraping. Ajouter un délai entre les requêtes. Une API officielle existe dans 70% des cas – cherche-la avant de scraper.

Extraire le texte d'un PDF :

```
import pdfplumber
with pdfplumber.open("rapport.pdf") as pdf:
    texte = "\n".join(p.extract_text() or ""
                      for p in pdf.pages)
```

Extraire un tableau d'un PDF :

```
with pdfplumber.open("tableau.pdf") as pdf:
    table = pdf.pages[0].extract_table()
df = pd.DataFrame(table[1:], columns=table[0])
```

Extraire depuis un Word :

```
from docx import Document
doc = Document("note.docx")
texte = "\n".join(p.text for p in doc.paragraphs)
```

Lire des emails IMAP :

```
import imaplib, email
mail = imaplib.IMAP4_SSL("imap.gmail.com")
mail.login(os.getenv("EMAIL"), os.getenv("PASS"))
mail.select("inbox")
_, ids = mail.search(None, "UNSEEN")
```

CHAPITRE II – EXPLORATION INITIALE

Dimensions et structure :

```
print(df.shape)          # (lignes, colonnes)
print(df.dtypes)         # type de chaque colonne
print(df.info())         # types + taux de remplissage
print(df.describe(include="all")) # stats par colonne
```

Valeurs manquantes :

```
missing = pd.DataFrame({
    "count": df.isnull().sum(),
    "pct":   df.isnull().mean().mul(100).round(1)
}).sort_values("pct", ascending=False)
print(missing[missing["count"] > 0])
```

Doublons :

```
print(f"Doublons exacts      : {df.duplicated().sum()}")
print(f"Doublons sur id     : {df.duplicated('id').sum()}")
doublons = df[df.duplicated(keep=False)].sort_values("id")
```

Distribution des catégorielles :

```
for col in df.select_dtypes("object").columns:
    print(f"\n{col} – {df[col].nunique()} valeurs uniques")
    print(df[col].value_counts().head(10))
```

Distribution des numériques :

```
print(df.describe(percentiles=[.05, .1, .25, .5, .75, .9, .95, .99]))
```

Cardinalité par colonne (utile pour détecter les clés) :

```
df.nunique().sort_values(ascending=False)
```

Mémoire utilisée :

```
print(df.memory_usage(deep=True).sum() / 1e6, "MB")
```

CHAPITRE III – NETTOYAGE COMPLET

```

Supprimer les colonnes trop vides (> 50% manquants) :
thresh = int(len(df) * 0.5)
df = df.dropna(thresh=thresh, axis=1)

Supprimer les lignes sans clé fonctionnelle :
df = df.dropna(subset=["id", "date", "montant"])

Remplacer selon le type et la logique métier :
df["montant"] = df["montant"].fillna(0)
df["categorie"] = df["categorie"].fillna("INCONNU")
df["prix"] = df["prix"].fillna(df["prix"].median())
df["region"] = df["region"].fillna(df["region"].mode()[0])

Interpolation pour les séries temporelles :
df["valeur"] = df["valeur"].interpolate(method="time")

Signaler les remplacements sans les cacher :
df["montant_imputed"] = df["montant"].isna()
df["montant"] = df["montant"].fillna(0)

```

— DOUBLONS —

```

Supprimer les doublons exacts :
df = df.drop_duplicates()

Supprimer les doublons sur clé en gardant le plus récent :
df = (df.sort_values("date", ascending=False)
      .drop_duplicates(subset=["id"])
      .sort_index())

Fusionner les doublons en agrégeant :
df = df.groupby("id").agg({
    "montant": "sum",
    "date": "max",
    "statut": "first"
}).reset_index()

```

— TYPES ET CONVERSIONS —

```

Dates avec gestion des formats multiples :
df["date"] = pd.to_datetime(df["date"],
    dayfirst=True, errors="coerce")
nb_invalides = df["date"].isna().sum()
print(f"{nb_invalides} dates invalides converties en NaT")

Numériques avec nettoyage préalable des chaînes :
df["montant"] = (df["montant"].astype(str)
    .str.replace(r"€\$", "", regex=True)
    .str.replace(",", ".", regex=False)
    .pipe(pd.to_numeric, errors="coerce"))

Catégorielles pour économiser la mémoire :
for col in ["statut", "region", "categorie"]:
    df[col] = df[col].astype("category")

Booléens depuis du texte :
mapping_bool = {"oui": True, "non": False,
    "yes": True, "no": False, "1": True, "0": False}
df["actif"] = df["actif"].str.lower().map(mapping_bool)

```

— NETTOYAGE DES TEXTES —

```

Pipeline de nettoyage standard :
def clean_text(series):
    return (series.astype(str)
        .str.strip()
        .str.upper()
        .str.replace(r"\s+", " ", regex=True)
        .str.replace(r"^\w\s", "", regex=True))

df["nom"] = clean_text(df["nom"])
df["ville"] = clean_text(df["ville"])

```

```

Harmoniser les variantes d'une catégorie :
corrections = {
    "PARIS 1": "PARIS", "PARIS ": "PARIS",
    "LYON CEDEX": "LYON", "ST ETIENNE": "SAINT-ETIENNE"
}
df["ville"] = df["ville"].replace(corrections)

```

```

Extraire des informations d'un texte :
df["annee"] = df["ref"].str.extract(r"(\d{4})")
df["code_region"] = df["code_postal"].str[:2]
df["domaine_mail"] = df["email"].str.split("@").str[1]

```

— OUTLIERS —

```

Méthode IQR – robuste et recommandée :
def flag_outliers_iqr(series, factor=1.5):
    Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
    IQR = Q3 - Q1
    return (series < Q1 - factor * IQR) | \
        (series > Q3 + factor * IQR)

```

```

df["outlier_montant"] = flag_outliers_iqr(df["montant"])
print(f"{df['outlier_montant'].sum()} outliers détectés")

```

```

Méthode Z-score – sensible aux données non normales :
from scipy import stats
df["zscore"] = np.abs(stats.zscore(df["montant"].dropna()))
df["outlier_z"] = df["zscore"] > 3

```

```

Winsorisation – plafonner plutôt que supprimer :
from scipy.stats import mstats
df["montant_wins"] = mstats.winsorize(
    df["montant"], limits=[0.01, 0.01])

```

Ne jamais supprimer un outlier sans validation métier.
Toujours flaguer. Laisser le client décider de l'action.

— TRANSFORMATION ET FEATURE ENGINEERING —

```

Dérivées temporelles utiles :
df["annee"] = df["date"].dt.year
df["mois"] = df["date"].dt.month
df["trimestre"] = df["date"].dt.quarter
df["semaine"] = df["date"].dt.isocalendar().week
df["jour_sem"] = df["date"].dt.day_name()
df["is_weekend"] = df["date"].dt.dayofweek >= 5
df["nb_jours_client"] = (
    pd.Timestamp.now() - df["premiere_commande"]).dt.days

```

```

Discrétisation (tranches) :
df["tranche_ca"] = pd.cut(df["ca"],
    bins=[0, 1000, 5000, 20000, np.inf],
    labels=["<1k", "1-5k", "5-20k", ">20k"])

```

```

df["quartile"] = pd.qcut(df["montant"], q=4,
    labels=["Q1", "Q2", "Q3", "Q4"])

```

```

Normalisation pour la comparaison :
from sklearn.preprocessing import MinMaxScaler
df["montant_norm"] = MinMaxScaler().fit_transform(
    df[["montant"]])

```

```

Agrégation et enrichissement :
stats_client = df.groupby("client_id").agg(
    nb_commandes = ("id", "count"),
    ca_total = ("montant", "sum"),
    panier_moyen = ("montant", "mean"),
    recence = ("date", lambda x:
        (pd.Timestamp.now() - x.max()).days)
).reset_index()
df = df.merge(stats_client, on="client_id", how="left")

```

```

Jointures avec vérification de cardinalité :
n_avant = len(df)
df = df.merge(ref_table, on="code", how="left")
n_apres = len(df)
if n_apres != n_avant:
    print(f"ALERTE : {n_apres-n_avant} lignes ajoutées !")

```

CHAPITRE IV – STATISTIQUES COMPLÈTES

— STATISTIQUES DESCRIPTIVES AVANCÉES —

```

Résumé complet d'une variable :
from scipy import stats as sc
col = df["montant"].dropna()
resume = {
    "n" : len(col),
    "moyenne" : col.mean(),
    "mediane" : col.median(),
    "ecart_type" : col.std(),
    "cv" : col.std() / col.mean(),
    "skewness" : sc.skew(col),
    "kurtosis" : sc.kurtosis(col),
    "p5" : col.quantile(0.05),
    "p25" : col.quantile(0.25),
    "p75" : col.quantile(0.75),
    "p95" : col.quantile(0.95),
    "iqr" : col.quantile(0.75) - col.quantile(0.25)
}

```

```

Matrice de corrélation complète :
corr = df.select_dtypes("number").corr(method="pearson")
corr_spearman = df.select_dtypes("number").corr(method="spearman")

```

```

Corrélations avec une cible triées par force :
target = "ca_total"
top_corr = corr[target].abs().sort_values(
    ascending=False).drop(target)

```

— TESTS STATISTIQUES —

```

Test de normalité (Shapiro-Wilk sur petit échantillon) :
stat, p = sc.shapiro(df["montant"].sample(min(50, len(df))))
print(f"Normal : {p > 0.05} (p={p:.4f})")

```

```

Test de normalité (Kolmogorov-Smirnov sur grand échantillon) :
stat, p = sc.kstest(sc.zscore(df["montant"].dropna()), "norm")
print(f"Normal : {p > 0.05} (p={p:.4f})")

```

```

Comparaison de deux moyennes (t-test) :
grp_A = df[df["segment"]=="A"]["montant"]
grp_B = df[df["segment"]=="B"]["montant"]
stat, p = sc.ttest_ind(grp_A, grp_B, equal_var=False)
print(f"Diff significative : {p < 0.05} (p={p:.4f})")

```

```

Comparaison de plusieurs groupes (ANOVA) :
groupes = [g["montant"].values
            for _, g in df.groupby("region")]
stat, p = sc.f_oneway(*groupes)

```

```

Test du chi-deux (indépendance de deux catégorielles) :
tableau = pd.crosstab(df["segment"], df["statut"])
stat, p, dof, expected = sc.chi2_contingency(tableau)

```

```

Test de Mann-Whitney (non paramétrique) :
stat, p = sc.mannwhitneyu(grp_A, grp_B,
    alternative="two-sided")

```

```

Intervalle de confiance à 95% pour une moyenne :
n, mean, sem = len(col), col.mean(), sc.sem(col)
ci = sc.t.interval(0.95, df=n-1, loc=mean, scale=sem)

```

```
print(f"IC 95% : [{ci[0]:.1f}, {ci[1]:.1f}]")

Régression linéaire simple :
slope, intercept, r, p, se = sc.linregress(
    df["budget_pub"], df["ca"])
print(f"R²={r**2:.3f} | pente={slope:.3f} | p={p:.4f}")
```

— ANALYSE TEMPORELLE —

```
Préparer la série temporelle :
ts = df.set_index("date").sort_index()

Resampling vers différentes granularités :
daily      = ts.resample("D").sum()
weekly     = ts.resample("W").sum()
monthly    = ts.resample("ME").sum()
quarterly  = ts.resample("QE").sum()
yearly     = ts.resample("YE").sum()

Moyennes mobiles simple et exponentielle :
monthly["mm3"] = monthly.rolling(3).mean()
monthly["mm12"] = monthly.rolling(12).mean()
monthly["mme"] = monthly.ewm(span=3).mean()

Taux de variation :
monthly["var_mois"] = monthly["montant"].pct_change()*100
monthly["var_annee"] = monthly["montant"].pct_change(12)*100
monthly["cumul_ytd"] = monthly["montant"].groupby(
    monthly.index.year).cumsum()

CAGR sur une période :
n_ans = (monthly.index[-1] - monthly.index[0]).days / 365
cagr = (monthly.iloc[-1] / monthly.iloc[0]) ** (1/n_ans) - 1
print(f"CAGR : {cagr*100:.1f}%")

Décomposition de la saisonnalité :
df.groupby(ts.index.month).mean() # par mois
df.groupby(ts.index.dayofweek).mean() # par jour semaine

Détection d'anomalies temporelles :
roll_mean = ts.rolling(30).mean()
roll_std = ts.rolling(30).std()
anomalies = ts[(ts > roll_mean + 2*roll_std) |
               (ts < roll_mean - 2*roll_std)]
```

CHAPITRE V – TOUS LES GRAPHIQUES

— CONFIGURATION GLOBALE —

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

BLEU    = "#2563EB"
ROUGE   = "#DC2626"
VERT    = "#16A34A"
ORANGE  = "#EA580C"
GRIS    = "#9CA3AF"
BG      = "#F9FAFB"

plt.rcParams.update({
    "figure.figsize"      : (12, 6),
    "figure.facecolor"    : BG,
    "axes.facecolor"      : BG,
    "axes.spines.top"     : False,
    "axes.spines.right"   : False,
    "axes.grid"           : True,
    "grid.alpha"          : 0.3,
    "font.family"         : "sans-serif",
    "axes.titlesize"      : 14,
```

```

    "axes.titleweight" : "bold",
    "axes.titlepad"    : 12,
    "axes.labelsize"   : 11,
})

def save(nom, dpi=150):
    plt.tight_layout()
    plt.savefig(f"output/{nom}.png", dpi=dpi,
                bbox_inches="tight", facecolor=BG)
    plt.close()

```

— GRAPHIQUES DE TENDANCE ET D'ÉVOLUTION —

COURBE SIMPLE

```

fig, ax = plt.subplots()
ax.plot(monthly.index, monthly["montant"]/1000,
        color=BLEU, linewidth=2.5, marker="o",
        markersize=4, label="CA mensuel")
ax.fill_between(monthly.index, monthly["montant"]/1000,
                alpha=0.1, color=BLEU)
ax.set_title("Évolution du CA – Tendence sur 24 mois")
ax.set_ylabel("CA (k€)")
ax.yaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda x, _: f"{x:.0f}k€"))
save("courbe_ca")

```

COURBE MULTI-SÉRIES (comparaison régions)

```

fig, ax = plt.subplots()
colors = [BLEU, ROUGE, VERT, ORANGE]
for (region, grp), color in zip(
    df.groupby("region"), colors):
    serie = grp.set_index("date")["montant"].resample("M").sum()
    ax.plot(serie.index, serie/1000, label=region,
            color=color, linewidth=2)
ax.legend(frameon=False)
ax.set_title("CA mensuel par région – Comparaison N vs N-1")
save("courbe_multi")

```

COURBE AVEC BANDE DE CONFIANCE

```

fig, ax = plt.subplots()
ax.plot(monthly.index, monthly["mm12"]/1000,
        color=BLEU, linewidth=2.5, label="Moyenne mobile 12m")
ax.fill_between(monthly.index,
                (monthly["mm12"] - monthly["montant"].std())/1000,
                (monthly["mm12"] + monthly["montant"].std())/1000,
                alpha=0.15, color=BLEU, label="± 1 écart-type")
ax.set_title("Tendance lissée avec intervalle de variation")
ax.legend(frameon=False)
save("courbe_confiance")

```

COURBE AVEC ANNOTATIONS D'ÉVÉNEMENTS

```

fig, ax = plt.subplots()
ax.plot(monthly.index, monthly["montant"]/1000,
        color=BLEU, linewidth=2)
evenements = {
    "2024-03": ("Lancement v2", VERT),
    "2024-07": ("Crise supply", ROUGE)
}
for date_str, (label, color) in evenements.items():
    date = pd.Timestamp(date_str)
    val = monthly.loc[date, "montant"] / 1000
    ax.axvline(date, color=color, linestyle="--",
               alpha=0.7, linewidth=1.5)
    ax.annotate(label, xy=(date, val),
                xytext=(10, 10), textcoords="offset points",
                color=color, fontsize=9)
ax.set_title("Évolution du CA avec événements clés")
save("courbe_events")

```

— GRAPHIQUES DE COMPARAISON —

BARRES VERTICALES

```

fig, ax = plt.subplots()

```



```

mois_labels = monthly.index.strftime("%b %Y")
bars = ax.bar(mois_labels, monthly["montant"]/1000,
              color=BLEU, alpha=0.85, edgecolor="white", linewidth=0.5)
ax.bar_label(bars, fmt="%0fk€",
             padding=4, fontsize=8, color="#374151")
plt.xticks(rotation=45, ha="right")
ax.set_title("CA mensuel – Janvier à Décembre 2024")
save("barres_verticales")

```

```

BARRES HORIZONTALES (Top N)
top10 = df.nlargest(10, "ca_total")
fig, ax = plt.subplots(figsize=(10, 7))
colors_bar = [BLEU if i > 0 else ORANGE
               for i in range(len(top10))]
bars = ax.barh(top10["client"], top10["ca_total"]/1000,
               color=colors_bar, alpha=0.85, edgecolor="white")
ax.bar_label(bars, fmt="%0fk€", padding=4, fontsize=9)
ax.invert_yaxis()
ax.set_title("Top 10 clients par CA – Focus sur le leader")
ax.axvline(top10["ca_total"].mean()/1000,
           color=ROUGE, linestyle="--",
           linewidth=1.5, label=f"Moyenne")
ax.legend(frameon=False)
save("barres_horizontales")

```

```

BARRES GROUPÉES (comparaison N vs N-1)
x = np.arange(len(mois_labels))
width = 0.35
fig, ax = plt.subplots()
ax.bar(x - width/2, ca_2023/1000, width,
      label="2023", color=GRIS, alpha=0.7)
ax.bar(x + width/2, ca_2024/1000, width,
      label="2024", color=BLEU, alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(mois_labels, rotation=45, ha="right")
ax.legend(frameon=False)
ax.set_title("CA mensuel – 2024 vs 2023")
save("barres_groupees")

```

```

BARRES EMPILÉES (répartition par catégorie)
pivot = df.groupby(["mois", "categorie"])[["montant"]].sum()
pivot = pivot.unstack(fill_value=0)
pivot.plot(kind="bar", stacked=True, figsize=(12, 6),
          colormap="Blues", edgecolor="white", linewidth=0.3)
plt.title("Répartition du CA par catégorie et par mois")
plt.xlabel("")
plt.legend(title="Catégorie", frameon=False,
          bbox_to_anchor=(1, 1))
plt.xticks(rotation=45, ha="right")
save("barres_empilees")

```

```

WATERFALL (variation entre deux périodes)
items = ["CA Jan", "+Nouveaux", "+Upsell",
         "-Churn", "-Downsell", "CA Fév"]
vals = [100, 25, 15, -20, -8, 112]
cumuls = [0] + list(pd.Series(vals[:-1]).cumsum())
colors = [GRIS if i in [0, len(vals)-1]
          else VERT if v > 0 else ROUGE
          for i, v in enumerate(vals)]
fig, ax = plt.subplots()
ax.bar(items, vals, bottom=cumuls,
      color=colors, edgecolor="white", linewidth=0.5)
ax.set_title("Décomposition de l'évolution du CA – Jan→Fév")
ax.axhline(0, color="#374151", linewidth=0.5)
save("waterfall")

```

— GRAPHIQUES DE DISTRIBUTION —

```

HISTOGRAMME
fig, ax = plt.subplots()
ax.hist(df["montant"].dropna(), bins=40,
       color=BLEU, alpha=0.8, edgecolor="white", linewidth=0.5)
median = df["montant"].median()
mean = df["montant"].mean()

```

```

ax.axvline(median, color=ROUGE, linewidth=2,
           linestyle="--", label=f"Médiane : {median:,.0f}€")
ax.axvline(mean, color=ORANGE, linewidth=2,
           linestyle=":", label=f"Moyenne : {mean:,.0f}€")
ax.legend(frameon=False)
ax.set_title("Distribution des montants de commande")
ax.set_xlabel("Montant (€)")
save("histogramme")

```

```

BOXPLOT (comparaison de distributions)
fig, ax = plt.subplots()
segments = [df[df["segment"]==s]["montant"].dropna()
            for s in df["segment"].unique()]
bp = ax.boxplot(segments, patch_artist=True,
               labels=df["segment"].unique(), vert=True)
for patch, color in zip(bp["boxes"],
                       [BLEU, ROUGE, VERT, ORANGE]):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)
ax.set_title("Distribution des montants par segment")
ax.set_ylabel("Montant (€)")
save("boxplot")

```

```

VIOLIN PLOT (distribution + densité)
fig, ax = plt.subplots()
sns.violinplot(data=df, x="segment", y="montant",
               palette={"A": BLEU, "B": VERT, "C": ORANGE},
               inner="quartile", ax=ax)
ax.set_title("Distribution et densité par segment")
save("violin")

```

```

KDE – COURBE DE DENSITÉ
fig, ax = plt.subplots()
for segment, grp in df.groupby("segment"):
    grp["montant"].plot.kde(ax=ax, linewidth=2,
                           label=segment)
ax.set_title("Densité de probabilité des montants par segment")
ax.set_xlabel("Montant (€)")
ax.legend(frameon=False)
ax.axvline(df["montant"].median(), color=ROUGE,
           linestyle="--", linewidth=1, alpha=0.7)
save("kde")

```

```

QQ-PLOT (test visuel de normalité)
from scipy import stats as sc
fig, ax = plt.subplots(figsize=(7, 7))
sc.probplot(df["montant"].dropna(), dist="norm", plot=ax)
ax.set_title("Q-Q Plot – Test visuel de normalité")
save("qqplot")

```

— GRAPHIQUES DE RELATION —

```

NUAGE DE POINTS (scatter)
fig, ax = plt.subplots()
scatter = ax.scatter(df["budget_pub"], df["ca"],
                    c=df["taux_conversion"], cmap="Blues",
                    alpha=0.6, edgecolor="white", linewidth=0.3,
                    s=df["nb_clients"]/10)
plt.colorbar(scatter, ax=ax, label="Taux de conversion")
slope, intercept = np.polyfit(
    df["budget_pub"], df["ca"], 1)
x_line = np.linspace(df["budget_pub"].min(),
                     df["budget_pub"].max(), 100)
ax.plot(x_line, slope*x_line + intercept,
       color=ROUGE, linewidth=2, linestyle="--",
       label=f"R²={np.corrcoef(df['budget_pub'], df['ca'])[0,1]**2:.2f}")
ax.set_title("Budget publicitaire vs CA – Corrélation")
ax.legend(frameon=False)
save("scatter")

```

```

BUBBLE CHART (3 dimensions)
fig, ax = plt.subplots()
ax.scatter(df["recence"], df["frequence"],
          s=df["valeur"]/50, c=df["valeur"],

```

```

    cmap="Blues", alpha=0.7, edgecolor="white")
ax.set_xlabel("Récence (jours depuis dernier achat)")
ax.set_ylabel("Fréquence (nb commandes)")
ax.set_title("Segmentation RFM – Taille = Valeur client")
save("bubble")

```

HEATMAP DE CORRÉLATION

```

corr = df.select_dtypes("number").corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f",
            cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            square=True, linewidths=0.5,
            annot_kws={"size": 9}, ax=ax)
ax.set_title("Matrice de corrélation")
save("heatmap_corr")

```

HEATMAP CALENDRIER (valeur par jour/semaine)

```

pivot_cal = df.groupby(
    [df["date"].dt.isocalendar().week,
     df["date"].dt.day_name()])["montant"].sum().unstack()
fig, ax = plt.subplots(figsize=(14, 6))
sns.heatmap(pivot_cal, cmap="Blues", linewidths=0.5,
            annot=True, fmt=".0f", ax=ax)
ax.set_title("Heatmap des ventes par semaine et jour")
save("heatmap_calendrier")

```

PAIRPLOT (toutes les relations entre variables)

```

sns.pairplot(df[["montant", "recence", "frequence", "valeur"]],
            diag_kind="kde", plot_kws={"alpha":0.5, "color":BLEU},
            diag_kws={"color":BLEU})
plt.suptitle("Relations entre variables clés", y=1.02)
save("pairplot")

```

— GRAPHIQUES DE COMPOSITION —

CAMEMBERT (3 parts max)

```

fig, ax = plt.subplots(figsize=(8, 8))
vals = df.groupby("top3")["ca"].sum()
colors = [BLEU, GRIS, ORANGE]
wedges, texts, autotexts = ax.pie(
    vals, labels=vals.index, colors=colors,
    autopct="%1.1f%%", pctdistance=0.8,
    startangle=90, wedgeprops={"edgecolor":"white",
    "linewidth":2})
for at in autotexts:
    at.set_fontsize(11)
    at.set_fontweight("bold")
ax.set_title("Répartition du CA – Top 3 segments")
save("camembert")

```

DONUT (anneau)

```

fig, ax = plt.subplots(figsize=(8, 8))
ax.pie(vals, labels=vals.index, colors=colors,
        autopct="%1.1f%%", pctdistance=0.85,
        startangle=90,
        wedgeprops={"width":0.5, "edgecolor":"white"})
ax.set_title("Répartition du CA")
save("donut")

```

TREEMAP (avec squarify)

```

import squarify
fig, ax = plt.subplots(figsize=(12, 7))
squarify.plot(
    sizes=df.groupby("client")["ca"].sum().values,
    label=df.groupby("client")["ca"].sum().index,
    color=sns.color_palette("Blues", len(df["client"].unique()))),
    alpha=0.85, ax=ax, pad=True, text_kwargs={"fontsize":9})
ax.set_title("Treemap du CA par client")
ax.axis("off")
save("treemap")

```

COURBE DE PARETO (règle des 80/20)

```

df_sorted = df.sort_values("ca", ascending=False)

```

```

df_sorted["cum_pct"] = (df_sorted["ca"].cumsum()
                        / df_sorted["ca"].sum() * 100)
df_sorted["rank_pct"] = np.arange(1, len(df_sorted)+1) \
                        / len(df_sorted) * 100

fig, ax1 = plt.subplots()
ax2 = ax1.twinx()
ax1.bar(range(len(df_sorted)), df_sorted["ca"]/1000,
        color=BLEU, alpha=0.7)
ax2.plot(range(len(df_sorted)), df_sorted["cum_pct"],
        color=ROUGE, linewidth=2)
ax2.axhline(80, color=ROUGE, linestyle="--",
            linewidth=1, alpha=0.5)
ax2.axvline(df_sorted[df_sorted["cum_pct"]<=80].index[-1],
            color=ORANGE, linestyle="--", linewidth=1)
ax1.set_title("Courbe de Pareto – Concentration du CA")
ax1.set_ylabel("CA (k€)")
ax2.set_ylabel("CA cumulé (%)")
save("pareto")

```

— GRAPHIQUES TEMPORELS AVANCÉS —

```

AIRE EMPILÉE (évolution de la composition)
pivot = df.groupby(["mois", "categorie"])["montant"].sum()
pivot = pivot.unstack(fill_value=0)
fig, ax = plt.subplots()
ax.stackplot(pivot.index, pivot.T,
            labels=pivot.columns,
            colors=[BLEU, VERT, ORANGE, ROUGE],
            alpha=0.8)
ax.legend(loc="upper left", frameon=False)
ax.set_title("Évolution de la composition du CA")
save("aire_empilee")

```

```

GRAPHIQUE EN CHANDELLES (candlestick OHLC)
import matplotlib.dates as mdates
fig, ax = plt.subplots()
for idx, row in ohlc.iterrows():
    color = VERT if row["close"] >= row["open"] else ROUGE
    ax.bar(idx, row["close"]-row["open"],
           bottom=row["open"], color=color, width=0.6)
    ax.plot([idx, idx], [row["low"], row["high"]],
           color=color, linewidth=1)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
ax.set_title("Évolution mensuelle OHLC")
save("candlestick")

```

```

GRAPHIQUE DE RUPTURE DE TENDANCE
fig, ax = plt.subplots()
point_rupture = pd.Timestamp("2024-06-01")
avant = monthly[monthly.index < point_rupture]
apres = monthly[monthly.index >= point_rupture]
ax.plot(avant.index, avant["montant"]/1000,
        color=BLEU, linewidth=2.5, label="Tendance avant")
ax.plot(apres.index, apres["montant"]/1000,
        color=ROUGE, linewidth=2.5, label="Tendance après")
ax.axvline(point_rupture, color=ORANGE,
            linestyle="--", linewidth=2, label="Rupture")
ax.legend(frameon=False)
ax.set_title("Détection visuelle d'une rupture de tendance")
save("rupture_tendance")

```

— GRAPHIQUES DE PERFORMANCE (STYLE BI) —

```

GAUGE / JAUGE (% d'objectif atteint)
import matplotlib.patches as mpatches
valeur = 73 # % atteint
fig, ax = plt.subplots(figsize=(8, 5))
theta = np.linspace(0, np.pi, 200)
for r, color in [(1.4, "#E5E7EB"), (1.3, "#E5E7EB")]:
    ax.plot(r*np.cos(theta), r*np.sin(theta), color=color, lw=15)
theta_val = np.linspace(0, np.pi*valeur/100, 200)
ax.plot(1.35*np.cos(theta_val), 1.35*np.sin(theta_val),

```

```

        color=VERT if valeur>=80 else ORANGE if valeur>=60
        else ROUGE, lw=15)
ax.text(0, 0.3, f"{valeur}%", ha="center", va="center",
        fontsize=40, fontweight="bold", color="#111827")
ax.text(0, -0.1, "Objectif CA", ha="center",
        fontsize=12, color="#6B7280")
ax.set_xlim(-1.6, 1.6)
ax.set_ylim(-0.3, 1.6)
ax.axis("off")
save("gauge")

```

```

BULLET CHART (performance vs objectif)
fig, ax = plt.subplots(figsize=(10, 4))
categories = ["CA", "Marge", "NPS", "Rétention"]
values = [87, 72, 61, 90]
targets = [80, 80, 70, 85]
y = np.arange(len(categories))
ax.barh(y, [100]*len(categories), height=0.5,
        color="#E5E7EB")
ax.barh(y, values, height=0.4,
        color=[VERT if v>=t else ORANGE if v>=t*0.85
               else ROUGE for v, t in zip(values, targets)])
for i, (v, t) in enumerate(zip(values, targets)):
    ax.plot([t, t], [i-0.35, i+0.35],
            color="#111827", linewidth=3)
ax.set_yticks(y)
ax.set_yticklabels(categories)
ax.set_xlim(0, 110)
ax.set_title("Performance vs Objectifs")
save("bullet_chart")

```

```

SPARKLINES (tendances compactes pour tableaux)
fig, axes = plt.subplots(4, 1, figsize=(6, 4))
metriques = ["CA", "Clients", "Panier moyen", "Churn %"]
for ax, metrique in zip(axes, metriques):
    serie = np.random.randn(12).cumsum() + 100
    ax.plot(serie, color=BLEU if serie[-1]>serie[0]
            else ROUGE, linewidth=1.5)
    ax.axis("off")
    ax.set_title(metrique, loc="left", fontsize=9,
                color="#6B7280")
plt.suptitle("Sparklines – Vue synthétique", fontsize=11)
save("sparklines")

```

— GRAPHIQUES GÉOGRAPHIQUES —

```

CARTE CHOROPLÈTHE AVEC GEOPANDAS (si données géo)
import geopandas as gpd
france = gpd.read_file("departements.geojson")
france = france.merge(df_regions, on="code_dept", how="left")
fig, ax = plt.subplots(figsize=(10, 12))
france.plot(column="ca_total", cmap="Blues",
            legend=True, missing_kwds={"color": "#F3F4F6"},
            edgecolor="white", linewidth=0.3, ax=ax)
ax.set_title("CA par département")
ax.axis("off")
save("carte_choropleth")

```

— DASHBOARD MULTI-GRAPHIQUES —

```

La façon de combiner plusieurs graphiques en une seule figure :
fig = plt.figure(figsize=(18, 10))
fig.suptitle("Dashboard Performance Commerciale – S1 2024",
            fontsize=16, fontweight="bold", y=1.01)

```

```

# Disposition en grille
ax1 = fig.add_subplot(2, 3, 1) # haut gauche
ax2 = fig.add_subplot(2, 3, 2) # haut centre
ax3 = fig.add_subplot(2, 3, 3) # haut droite
ax4 = fig.add_subplot(2, 1, 2) # bas pleine largeur

```

```

# Chaque ax reçoit son graphique
ax1.bar(...) # KPI rapide

```

```
ax2.pie(...)      # répartition
ax3.plot(...)     # tendance
ax4.plot(...)     # évolution détaillée
```

```
plt.tight_layout()
save("dashboard_complet")
```

Pour un rendu interactif navigable en HTML :

```
import plotly.express as px
fig = px.line(monthly, x=monthly.index, y="montant",
              title="CA mensuel interactif")
fig.update_layout(template="plotly_white")
fig.write_html("dashboard.html")
```

CHAPITRE VI – RAPPORT AUTOMATIQUE

— GÉNÉRER UN RAPPORT EXCEL COMPLET —

```
with pd.ExcelWriter("rapport.xlsx", engine="openpyxl") as w:
    df_kpi.to_excel(w, sheet_name="KPIs", index=False)
    df_ca.to_excel(w, sheet_name="CA", index=False)
    df_top.to_excel(w, sheet_name="Top clients", index=False)
    df_raw.to_excel(w, sheet_name="Données", index=False)
```

Insérer les graphiques dans Excel :

```
from openpyxl import load_workbook
from openpyxl.drawing.image import Image as XLImage
wb = load_workbook("rapport.xlsx")
ws = wb["KPIs"]
img = XLImage("output/courbe_ca.png")
img.anchor = "A20"
ws.add_image(img)
wb.save("rapport_final.xlsx")
```

— GÉNÉRER UN RAPPORT PDF —

```
from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.platypus import (SimpleDocTemplate, Paragraph,
                                Spacer, Image, Table, TableStyle)
from reportlab.lib.styles import getSampleStyleSheet

doc = SimpleDocTemplate("rapport.pdf", pagesize=A4)
story = []
styles = getSampleStyleSheet()

story.append(Paragraph(
    "Rapport d'Analyse – Janvier 2024", styles["Title"]))
story.append(Spacer(1, 20))

story.append(Paragraph("1. Synthèse", styles["Heading1"]))
story.append(Paragraph(
    f"CA total : {ca_total:,.0f} €", styles["Normal"]))

story.append(Image("output/courbe_ca.png",
    width=450, height=220))

data = [["Segment", "CA", "Nb clients", "Panier moyen"]] + \
    df_kpi.values.tolist()
t = Table(data, hAlign="LEFT")
t.setStyle(TableStyle([
    ("BACKGROUND", (0,0), (-1,0), colors.HexColor("#2563EB")),
    ("TEXTCOLOR", (0,0), (-1,0), colors.white),
    ("FONTSIZE", (0,0), (-1,-1), 9),
    ("ROWBACKGROUNDS", (0,1), (-1,-1),
        [colors.white, colors.HexColor("#F3F4F6")]),
    ("GRID", (0,0), (-1,-1), 0.3, colors.grey)
]))
story.append(t)
doc.build(story)
```

```

from jinja2 import Template
import base64, io

def fig_to_base64(fig):
    buf = io.BytesIO()
    fig.savefig(buf, format="png", dpi=120,
                bbox_inches="tight")
    buf.seek(0)
    return base64.b64encode(buf.read()).decode()

template = Template("""
<!DOCTYPE html><html><body>
<h1>Rapport {{ date }}</h1>
<div style="display:flex; gap:20px">
  <div><h3>CA Total</h3><p>{{ ca_total }}</p></div>
  <div><h3>Clients actifs</h3><p>{{ nb_clients }}</p></div>
  <div><h3>Panier moyen</h3><p>{{ panier_moyen }}</p></div>
</div>

{{ tableau | safe }}
</body></html>
""")

html = template.render(
    date=pd.Timestamp.now().strftime("%d/%m/%Y"),
    ca_total=f"{ca_total:,.0f} €",
    nb_clients=nb_clients,
    panier_moyen=f"{panier_moyen:,.0f} €",
    graphique_ca=fig_to_base64(fig_ca),
    tableau=df_top10.to_html(index=False, classes="table")
)
with open("rapport.html", "w", encoding="utf-8") as f:
    f.write(html)

```

```

import streamlit as st

st.set_page_config(page_title="BI Dashboard",
                    layout="wide", page_icon="📊")
st.title("Dashboard Performance")

col1, col2, col3, col4 = st.columns(4)
col1.metric("CA Total", f"{ca_total:,.0f}€",
            delta=f"{var_ca:+.1f}%")
col2.metric("Clients", nb_clients,
            delta=f"{var_clients:+.0f}")
col3.metric("Panier moyen", f"{panier_moyen:,.0f}€")
col4.metric("Churn", f"{churn:.1f}%",
            delta=f"{var_churn:+.1f}%", delta_color="inverse")

st.divider()
col_g, col_d = st.columns([2, 1])

with col_g:
    st.subheader("Évolution du CA")
    st.line_chart(monthly.set_index("date")["montant"])

with col_d:
    st.subheader("Top 5 clients")
    st.dataframe(df_top5.style.bar(
        subset=["ca"], color=BLEU),
        use_container_width=True)

with st.expander("📄 Données détaillées"):
    st.dataframe(df, use_container_width=True)
    st.download_button("📄 Télécharger",
                        df.to_csv(index=False, "data.csv"))

```

La configuration globale de matplotlib vaut dix heures de mise en forme graphique par graphique. Configure une seule fois en début de script et tous tes graphiques sont cohérents.

Seaborn est matplotlib avec les bonnes valeurs par défaut. Utilise-le pour les distributions et les corrélations. Utilise matplotlib pour tout ce qui demande un contrôle fin. Utilise plotly pour tout ce qui doit être interactif.

Un dashboard à cinq métriques clés et trois graphiques bien choisis vaut mieux que vingt-cinq graphiques qui noient le décideur. La sélection est un travail analytique autant que technique.

Le graphique que le client imprimera et accrochera dans sa salle de réunion – c'est celui-là qui compte. Conçois pour ce cas d'usage, pas pour ton propre plaisir de dataviz.

Tout graphique livré au client doit avoir un titre qui est une conclusion, une source, une date, et une unité. Sans ces quatre éléments, le graphique peut être mal interprété dans six mois par quelqu'un qui n'était pas là.
